



Today's agenda

↳ GCD Intro

↳ Properties of gcd

↳ gcd function

↳ gcd problems

↳ Subsequence with gcd = 1

↳ Delete 1 element.

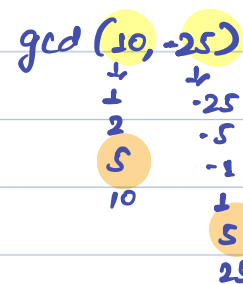
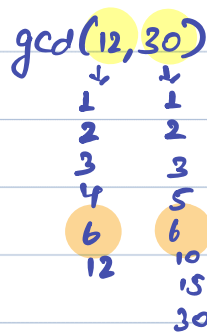
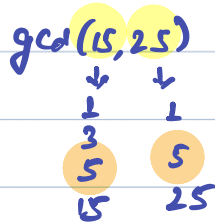


AlgoPrep

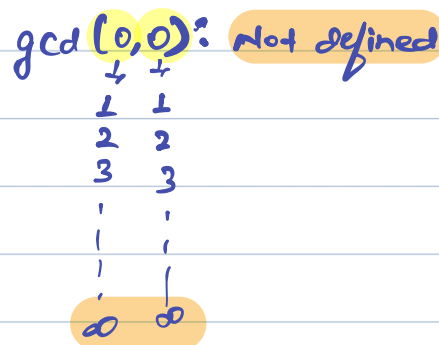
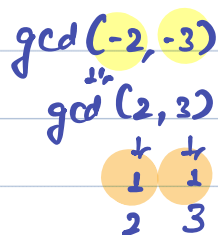
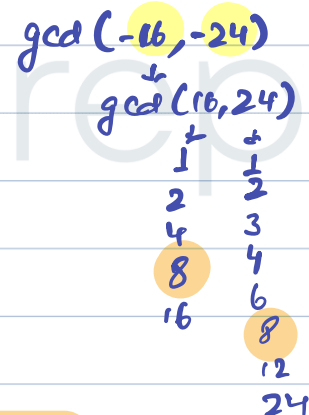
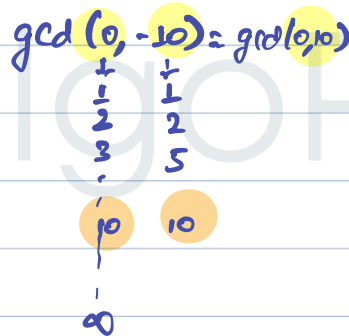
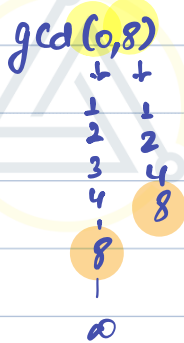


GCD: greatest common divisor / HCF $\rightarrow$ Highest common factor.

$\text{gcd}(a, b) : n \mid \{ n \text{ is greatest number, } a \% n == 0 \text{ and } b \% n == 0 \}$



GCD will always be +ve.





Properties of $\gcd(a, b)$

$$\hookrightarrow \gcd(a, b) = \gcd(b, a) \text{ } \} \text{Commutative Property}$$

$$\hookrightarrow \gcd(a, b) = \gcd(|a|, |b|)$$

$$\hookrightarrow \gcd(0, n) = |n|, \text{ if } n \neq 0$$

$$\hookrightarrow \gcd(a, b, c) = \gcd(\gcd(a, b), c) \text{ } \} \text{Associative Property}$$



AlgoPrep

→ Given 2 numbers a & b find gcd of them.



6 GCD

$$a < b$$
$$\text{gcd}(24, 34)$$

$$\begin{array}{r} 1 \\ 24 \overline{) 34} \\ \underline{24} \\ 10 \end{array}$$
$$\begin{array}{r} 2 \\ 10 \overline{) 24} \\ \underline{20} \\ 4 \end{array}$$
$$\begin{array}{r} 2 \\ 4 \overline{) 10} \\ \underline{8} \\ 2 \end{array}$$
$$\begin{array}{r} 2 \\ a \ 2 \overline{) 4 \ b} \\ \underline{4} \\ a \ 0 \end{array}$$
$$\begin{array}{r} 2 \\ a \ 0 \overline{) 2 \leftarrow b} \end{array}$$

$$\begin{aligned} \text{gcd}(24, 34) &= \text{gcd}(10, 24) = \text{gcd}(4, 10) \\ &= \text{gcd}(2, 4) = \text{gcd}(0, 2) \\ &= 2 \end{aligned}$$

$$\text{gcd}(a, b) = \text{gcd}(b \% a, a)$$

$$\begin{aligned} \text{gcd}(24, 34) &: \text{gcd}(10, 24) \\ &= \text{gcd}(4, 10) = \text{gcd}(2, 4) \end{aligned}$$

euclidean algo.



// Pseudo code

T.C: $\log_2 N$
 $\downarrow$
 $\max(a, b)$

S.C: $\log N$
 $\uparrow$
 stack space

```
int gcd (int a, int b) {
  if (a == 0) { return b; }
}
```

```
int temp = gcd (b % a, a)
return temp;
```

}

Tracing

$$\begin{aligned} \text{gcd}(24, 34) &= \text{gcd}(10, 24) = \text{gcd}(4, 10) \\ &= \text{gcd}(2, 4) = \text{gcd}(0, 2) \\ &= 2 \end{aligned}$$

```
int gcd (int a, int b) {
  1 if (a == 0) { return b; }
```

```
2 int temp = gcd (b % a, a)
```

```
3 return temp;
```

}

gcd {	a = 0 b = 2	1
2 {	a = 2 b = 4	1 2 3
2 {	a = 4 b = 10	1 2 3
2 {	a = 10 b = 24	1 2 3
2 {	a = 24 b = 34	1 2 3

2 {

$\text{gcd}(b \% a, a)$

$$\text{gcd}(34, 24) = \text{gcd}(24, 34)$$

$a > b \quad \rightarrow \quad a < b$



Q) Given $arr[N]$, calculate gcd of entire array.

Ex: $arr[3] = \{6, 12, 15\}$

$\downarrow 6 \rightarrow 3$

$arr[4] = \{8, 16, 12, 10\}$

$\downarrow 8 \rightarrow 4 \rightarrow 2$

// Pseudo code

```
int gcdarr (int arr[N]) {
```

```
    int ans = math.abs(arr[0]);
```

```
    for (int i = 1; i < N; i++) {
```

```
        ans = gcd(ans, math.abs(arr[i]));
```

```
    }
```

```
    return ans;
```

```
}
```

$\{8, 16, 12, 10\}$

$ans = 8 \rightarrow 4 \rightarrow 2$

T.C: $O(N \log N)$

S.C: $O(\log N)$

Space

$\hookrightarrow$ gcd function written in C++ Problem



→ **Subsequence**: you can select or reject numbers of the **array** in given order.

arr[4]: { 4 6 3 8 }
→ 4 8 ✓
→ 3 ✓
→ 8 6 ✗
→ 4 6 3 8 ✓

arr[2]: { 4 6 }

arr[3]: { 4 6 3 }

—	—
—	6
4	—
4	6

—	—	—
—	—	3
—	6	—
—	6	3
4	—	—
4	—	3
4	6	—
4	6	3

→ no. of subsequence of n length array: 2^n



Q) Given an array, check if there exists a subsequence with $\text{gcd} == 1$.

Ex: $\text{arr}[4] : \{4, 6, 3, 8\} \rightarrow \{4, 3\}, \{4, 3, 8\}$
↳ true

$\text{arr}[5] : \{16, 10, 6, 15, 27\} \rightarrow \{10, 27\}$
↳ true

$\text{arr}[4] : \{6, 12, 3, 18\} \rightarrow \text{false}$

||idea 1

↳ find all the subsequences and check for $\text{gcd} == 1$.

T.C: $2^n * \{\text{gcd of subsequences}\}$

XX ||idea 2 $\rightarrow$ if Prime is present with a different number, answer is true.

↳ The answer can be true even without prime numbers.

$\text{gcd}(9, 18) = 9 \rightarrow$ odd even combination doesn't work always.



idea 3

a b c d e f g h

Assume: $\gcd(b, e, h) = 1$

$\rightarrow \gcd(a, b, c, d, e, f, g, h)$

$\rightarrow \gcd(\cancel{\gcd(b, e, h)}, \cancel{\gcd(a, c, d, f, g)})$

$\rightarrow \gcd(1, \cancel{\gcd(a, c, d, f, g)}) = 1$

$\downarrow$
if you have a subsequence with $\gcd = 1$,
gcd of whole array is going to be 1.



//Pseudo code

```
boolean gcdSubs (int arr[N]) {  
    int ans = math.abs(arr[0]);  
    for (int i = 1; i < N; i++) {  
        ans = gcd(ans, math.abs(arr[i]));  
    }  
    if (ans == 1) { return true; }  
    else { return false; }  
}
```



Q) Delete to maximize

↳ Given $arr[N]$ elements, we have to delete 1 element
Such that gcd of remaining array is max.

Ex: $\begin{matrix} 0 & 1 & 2 & 3 & 4 \\ 24 & 16 & 18 & 30 & 15 \end{matrix}$
 $* [\quad] = 1$

$$\begin{matrix} 0 & 1 & 2 & 3 & 4 \\ 24 & 16 & 18 & 30 & 15 \end{matrix}$$
$$[] * [\quad] = 3$$

$\Rightarrow ans = 3$

$$\begin{matrix} 0 & 1 & 2 & 3 & 4 \\ 24 & 16 & 18 & 30 & 15 \end{matrix}$$
$$[\quad] * [\quad] = 1$$

$$\begin{matrix} 0 & 1 & 2 & 3 & 4 \\ 24 & 16 & 18 & 30 & 15 \end{matrix}$$
$$[\quad] * [\quad] = 1$$

$$\begin{matrix} 0 & 1 & 2 & 3 & 4 \\ 24 & 16 & 18 & 30 & 15 \end{matrix}$$
$$[\quad] * [\quad] = 2$$



//idea1

```
int ans = -1;  
for (int i=0; i<n; i++) {  
    //neglect arr[i] & find gcd of rem. element -> n  
    ans = mon(ans, arr[i]);  
}
```

T.C: $O(N * N \log N) = O(N^2 \log N)$

//idea2

	0	1	2	3	4 ✓
arr[5]:	24	16	18	30	15
Pgcd[5]:	24	8	2	2	1
Sgcd[5]:	1	1	3	15	15

$i=0 \rightarrow \text{sgcd}(i) = 1$

$i=1 \rightarrow$
left = $\text{pgcd}(i-1) = \text{pgcd}(0) = 24$
right = $\text{sgcd}(i+1) = \text{sgcd}(2) = 3$

$\text{gcd}(24, 3) = 3$

$i=2 \rightarrow$
left = $\text{pgcd}(i-1) = \text{pgcd}(1) = 1$
right = $\text{sgcd}(i+1) = \text{sgcd}(3) = 15$
 $\text{gcd}(1, 15) = 1$



// Pseudo code

T.C: $O(N \log N)$
S.C: $O(N)$

```
int deleteToMaximize (int arr[N]) {  
    int [ ] Pgcd = func1(arr);  $\rightarrow$  H.W  
    int [ ] Sgcd = func2(arr);  $\rightarrow$  will do  
    int ans = 0;  
    ans = Math.max(ans, Sgcd[0]);  
    for (int i = 1; i < N-1; i++) {  
        int left = Pgcd[i-1];  
        int right = Sgcd[i+1];  
        int overalgcd = gcd(left, right);  
        ans = Math.max(ans, overalgcd);  
    }  
    ans = Math.max(ans, Pgcd[N-2]);  
    return ans;  
}
```



```
int[] func2 (int arr[N]) {
```

```
    int[] sgcd = new int[N];
```

```
    sgcd[N-1] = arr[N-1];
```

```
    for (int i = N-2; i >= 0; i--) {
```

```
        sgcd[i] = gcd(sgcd[i+1], arr[i]);
```

```
    }
```

```
    return sgcd;
```

```
}
```



AlgoPrep

→ leetcode 2447 → {medium}



Q) Number of Subarrays with GCD equals K.

↳ Given $arr[N]$ and K , return the number of Subarrays of nums where the gcd equals K .

$K=3$

$arr[6]: \{ 9 \ 3 \ 1 \ 2 \ 6 \ 3 \} \rightarrow 4$

$\rightarrow \{3\} \quad \{3\}$

$\{9 \ 3\} \quad \{6 \ 3\}$

//idea 1

```
int count = 0;
for (int sp = 0; sp < n; sp++) {
    int ans = 0;
    for (int ep = sp; ep < n; ep++) {
        ans = gcd(ans, arr[ep]);
        if (ans < K) break;
        if (ans == K) count++;
    }
}
```

$K=3$

$arr[6]: \{ 9 \ 3 \ 1 \ 2 \ 6 \ 3 \}$

$ans: (0, 3) = (3, 1) = 1$

$count = 0 + 2$

T.C: $O(N^2 \cdot \log N)$

→ easy tag



Q) x of a kind in a deck

↳ you are given integer array deck where $deck[i]$ is a number. Partition the cards into one or more groups, i.e.

- Each group has exactly x card where $x \geq 1$.
- All cards in one group should have same integer.

ex1: 2 3 8 2 3 8 4 8 4 8 → true

ex2: 2 4 3 2 2 4 5 3 → false

//idea1

2	→	4
3	→	2 ✓
5	→	6
10	→	4

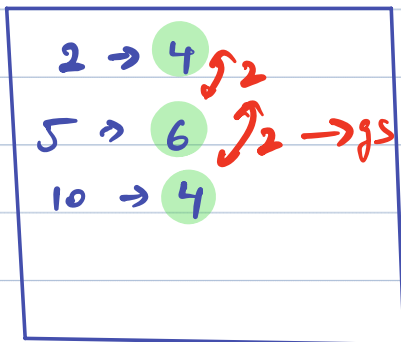
✓
| 3 3 | 2 2 | 2 2 |
| 5 5 | 5 5 | 5 5 | 10 10 |
10 10

↳ incorrect idea



//idea2

→ 2 2 2 2 5 5 5 5 5 5 10 10 10 10

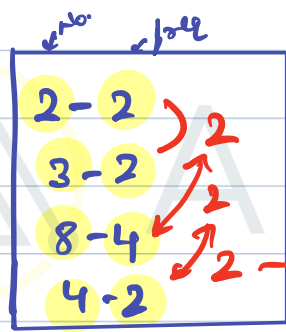


group size = 2 → gcd of occ.

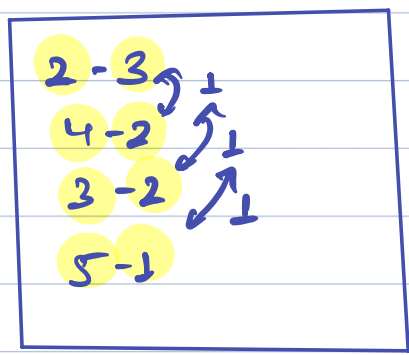
↳ gcd > 1 → true
↳ gcd == 1 → false

2 2 / 2 2 / 5 5 / 5 5 / 5 5 / 10 10 / 10 10

ex1: 2 3 8 2 3 8 4 8 4 8



ex2: 2 4 3 2 2 4 5 3



→ false



// Pseudo Code

boolean x of a kind (int arr[N]) {

HashMap<Integer, Integer> map = new HashMap;

for (int i=0; i<N; i++) {

if (map.containsKey(arr[i]) == false) {
map.put(arr[i], 1);

}

else {
int temp = map.get(arr[i]);
map.put(arr[i], temp+1);

}

}

int ans = 0;

for (int k: map.keySet())

ans = gcd(ans, map.get(k));

}

if (ans == 1) { return false; }
else { return true; }

}

T.C: $O(N \log N)$

S.C: $O(N)$



↳ you didn't come this far only to come this far.



AlgoPrep